СОДЕРЖАНИЕ

Введение 3

1 АЛГОРИТМ СОРТИРОВКИ 4

2 Подсчет сложности 5

3 Описание используемых переменных 6

4 Тестирование алгоритма 7

5 БЛОК-СХЕМА АЛГОРИТМА 8

6 ИСХОДНЫЙ КОД АЛГОРИТМА 9

Заключение 13

Приложение А 14

ВВЕДЕНИЕ

Целью работы является реализация алгоритма сортировки бинарными вставками для упорядочивания последовательности целых чисел, считанных из внешнего текстового файла. Программа должна поддерживать два режима работы с памятью: использование статического массива фиксированного размера и динамическое выделение памяти под точное количество элементов файла.

Основные задачи:

1. Реализовать функцию бинарного поиска для нахождения позиции вставки.
2. Реализовать алгоритм сортировки с минимизацией количества сравнений.
3. Организовать корректное чтение данных из файла и обработку выбора пользователя.
4. Провести теоретический анализ сложности алгоритма.

Программа реализуется на языке программирования C.

1 АЛГОРИТМ СОРТИРОВКИ

Алгоритм сортировки бинарными вставками является улучшенной версией стандартной сортировки вставками. Основное отличие заключается в использовании бинарного поиска для нахождения позиции вставки очередного элемента, что сокращает количество сравнений с $O(n)$ до $O(\log n)$.

Пошаговое описание:

1. **Разделение массива**: Массив делится на отсортированную (левую) и неотсортированную (правую) части. Изначально отсортированной считается первый элемент.
2. **Выбор ключа**: Берется первый элемент неотсортированной части (`selected`).
3. **Бинарный поиск**:
   - Устанавливаются границы `low = 0` и `high = i - 1`.
   - Пока `low <= high`, вычисляется `mid`.
   - Если `selected` больше элемента в `mid`, поиск продолжается в правой половине, иначе — в левой.
   - Функция возвращает позицию `loc`.
4. **Сдвиг элементов**: Все элементы от `loc` до текущего индекса сдвигаются на одну позицию вправо.
5. **Вставка**: `selected` помещается в освободившуюся позицию `loc`.

Процесс повторяется $n - 1$ раз.

2 Подсчет сложности

Для оценки сложности $O(n)$ рассмотрим количество элементарных операций (сравнения и копирования).

1. **Бинарный поиск**: На каждой итерации i выполняется $\log_2 i$ сравнений.
2. **Внешний цикл**: Выполняется $n - 1$ раз.
3. **Сдвиг элементов (худший случай)**: Если массив отсортирован в обратном порядке, на каждой итерации i выполняется i операций сдвига.

Итоговая сложность:

$$T(n) = \sum_{\{i=1\}}^{\{n-1\}} (\log_2 i + i) \tag{1}$$

Асимптотическая оценка:

- **Количество сравнений**: $O(n \log n)$.
- **Количество пересылок (сдвигов)**: $O(n^2)$.
- **Общая сложность**: В худшем и среднем случае — $O(n^2)$, так как доминирующим фактором является сдвиг элементов в массиве.

3 Описание используемых переменных

Таблица 1 — Используемые переменные

Название	Тип	Границы	Назначение
argc	int	≥ 1	Количество аргументов командной строки
argv	char**	адрес	Массив строк аргументов командной строки
filename	char*	адрес	Указатель на имя входного файла
choice	int	0, 1, 2	Флаг выбора способа выделения памяти
file	FILE*	адрес	Указатель на файловый поток
n / count	unsigned int	$0..2^{32}-1$	Общее количество чисел в файле
static_arr	int[1000]	-	Статический массив для хранения чисел
arr / a	int*	адрес	Указатель на динамический массив или массив-параметр
i, j	int	$0..n$	Индексы для циклов и итерации по массивам
item / selected / temp / num	int	$-2^{31}..2^{31}-1$	Временное хранение текущего числа для сравнения/вставки
l, r	int	$0..n$	Левая и правая границы в бинарном поиске
mid	int	$0..n$	Индекс середины отрезка в бинарном поиске
loc	int	$0..n$	Найденный индекс для вставки элемента

4 Тестирование алгоритма

```
$> ./main parts.txt static
Отсортированный массив:
6 67 145 456 8998
```

Рисунок 1 — Результат успешной работы алгоритма (статический массив)

```
$> ./main input.txt dynamic
Отсортированный массив:
-32 40 123 152 777 1000 43283
```

Рисунок 2 — Результат успешной работы алгоритма (динамический массив)

```
$> ./main a static
Ошибка открытия файла!
→ 1
```

Рисунок 3 — Обработка отсутствия файла

5 БЛОК-СХЕМА АЛГОРИТМА

Блок-схема алгоритма представлена в Приложении А.

6 ИСХОДНЫЙ КОД АЛГОРИТМА

```
#include <stdio.h>   // для работы с файлами и ввода/вывода
#include <stdlib.h>  // для динамического выделения памяти
#include <string.h>  // для работы со строками

#define MAX_STATIC_SIZE 1000

// Функция бинарного поиска места для вставки
// Сложность: O(log n)
int binarySearch(int a[], int item, int l, int r) {
    while (l <= r) {            // (13 * log n)
        int mid = (l + r) / 2;  // (3)

        if (item == a[mid])  // (2)
            return mid + 1;  // (2)

        if (item > a[mid])  // (2)
            l = mid + 1;    // (2)
        else
            r = mid - 1;  // (2)
    }  // (1 + 4 + 2 + 2 + 2 + 2 = 13)
    return l;  // (1)
}

// Функция сортировки бинарными вставками
// Сложность: O(n^2)
void binaryInsertionSort(int a[], int n) {
    int i, loc, j, sel;  // (0)

    // ((1 + n + 2n - 2) + (n - 1) * (3n + 13 log n + 7) = 3n^2 + 7n
+ 13(n - 1) log(n) - 8)
    for (i = 1; i < n; ++i) {  // (2)
        j = i - 1;             // (2)
        sel = a[i];            // (2)

        // (13 log n)
        loc = binarySearch(a, sel, 0, j);

        // (6n) Сдвиг элементов
        while (j >= loc) {
            a[j + 1] = a[j];  // (4)
            --j;              // (2)
```

```
        }  // (1 + 4 + 1 = 6)

        a[j + 1] = sel;  // (3)
    }  // (2 + 2 + 2 + 13 log n + 6n + 3) = (3n + 13 log n + 7)
}

// Вспомогательная функция для подсчета количества чисел в файле
unsigned int countNumbers(FILE* file) {
    unsigned int count = 0;  // (1)
    int temp;                // (0)
    // (6m), где m - размер файла в байтах
    while (!feof(file)) {                      // (1)
        if (fscanf(file, "%d", &temp) == 1) {  // (2)
            count++;                           // (2)
        } else {
            fgetc(file);  // (1)
        }
    }  // (1 + 2 + 2 + 1 = 6)

    return count;
}

// Вспомогательная функция для чтения чисел из файла в массив
void readNumbers(FILE* file, int arr[], int n) {
    int num;
    int i = 0;
    // (8 * max(m, n)), где m - размер файла в байтах
    while (!feof(file) && i < n) {            // (2)
        if (fscanf(file, "%d", &num) == 1) {  // (2)
            arr[i++] = num;                   // (3)
        } else {
            fgetc(file);  // (1)
        }
    }  // (2 + 2 + 3 + 1 = 8)
}

int main(int argc, char* argv[]) {
    if (argc != 3)
{                                                        // (1)
        printf("Использование: %s <имя_файла> <static|dynamic>\n",
argv[0]);  // (2)
        return
1;                                                          // (1)
```

```
    }

    int choice;                       // (0)
    unsigned int n = 0;               // (1)
    int* arr = NULL;                  // (1)
    int static_arr[MAX_STATIC_SIZE];  // (0)
    char* filename = argv[1];         // (1)

    if (strcmp(argv[2], "static") == 0) {          // (3)
        choice = 1;                                // (1)
    } else if (strcmp(argv[2], "dynamic") == 0) {  // (3)
        choice = 2;                                // (1)
    } else {
        printf("Неверный выбор.
\n");                                          // (1)
        printf("Использование: %s <имя_файла> <static|dynamic>\n",
argv[0]);  // (1)
        return
1;                                                          // (1)
    }

    FILE* file = fopen(filename, "r");  // (2)
    if (file == NULL) {                 // (1)
        printf("Ошибка открытия файла!\n");
        return 1;
    }

    n = countNumbers(file);  // (6m), где m - размер файла в байтах
    rewind(file);            // (1)

    if (choice == 1) {  // (1)

        readNumbers(file, static_arr, n);  // (8 * max(n, m))

        binaryInsertionSort(static_arr, n);  // (3n^2 + 7n + 13(n -
1) log(n) - 8))

        printf("Отсортированный массив:\n");
        for (int i = 0; i < n; i++)        // ((2n + 2) + (2n) = 4n +
2)
            printf("%d ", static_arr[i]);  // (2)
    } else {
        arr = (int*)malloc(n * sizeof(int));  // (2)
```

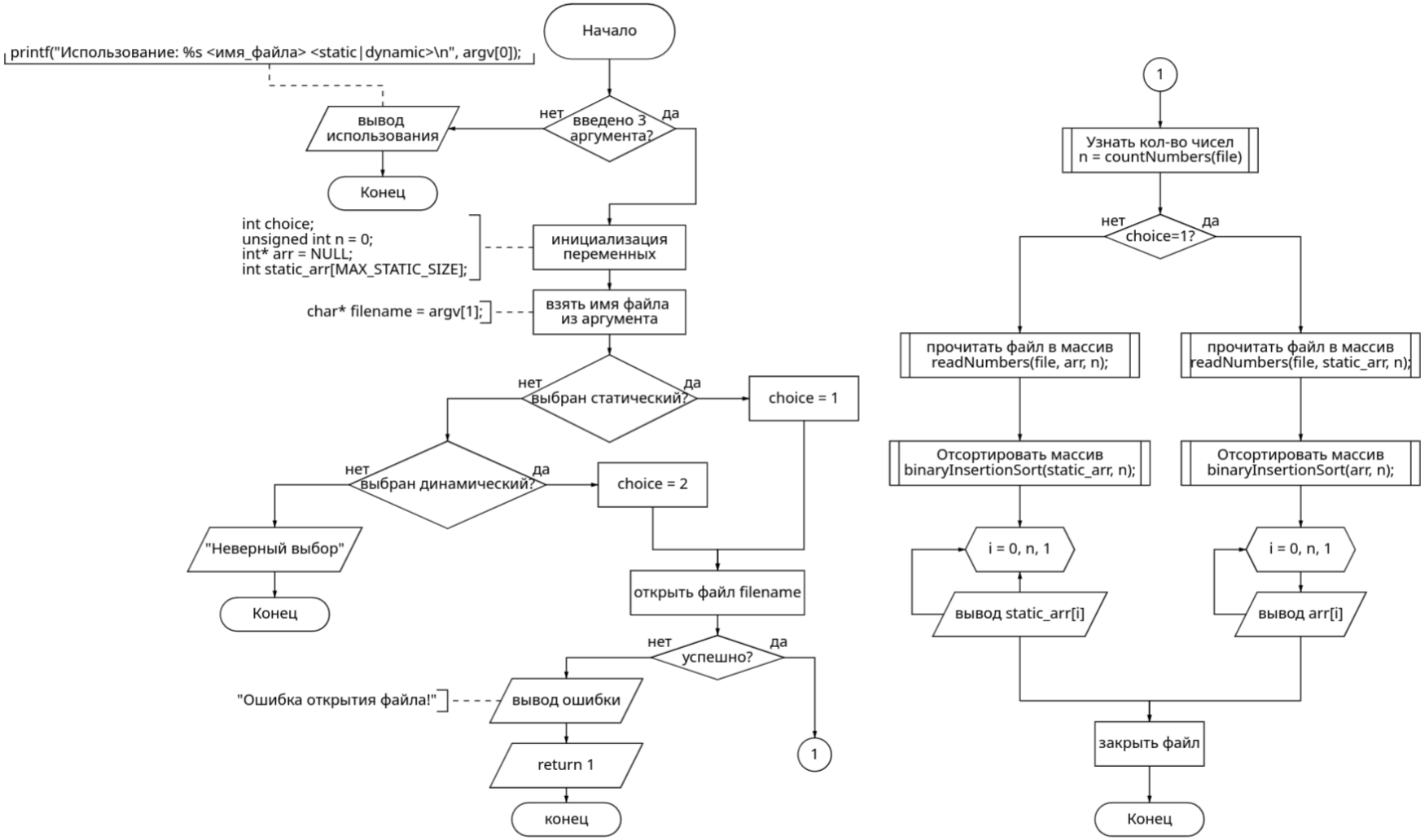
```
        if (arr == NULL) {                          // (1)
            printf("Ошибка выделения памяти!\n");  // (1)
            fclose(file);                           // (1)
            return 1;                               // (1)
        }

        readNumbers(file, arr, n);  // (8 * max(n, m))

        binaryInsertionSort(arr, n);  // (3n^2 + 7n + 13(n - 1)
log(n) - 8))

        printf("Отсортированный массив:\n");
        for (int i = 0; i < n; i++) printf("%d ", arr[i]);

        free(arr);  // (1)
    }
    printf("\n");
    fclose(file);  // (1)
    return 0;      // (1)
}
```

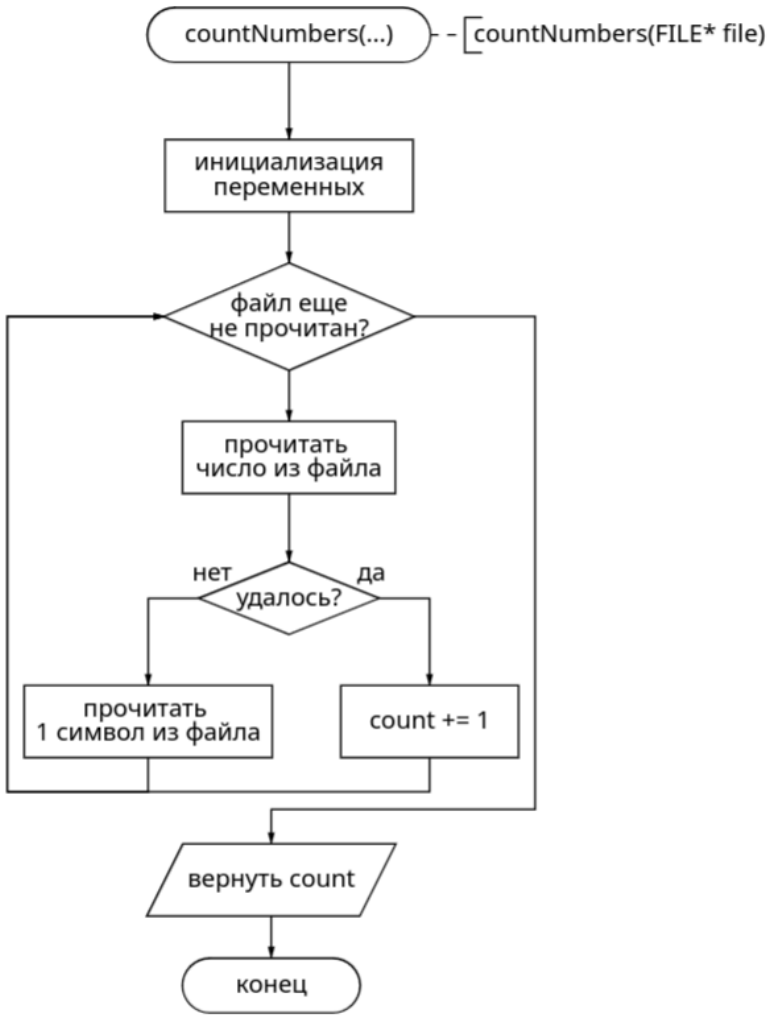
ЗАКЛЮЧЕНИЕ

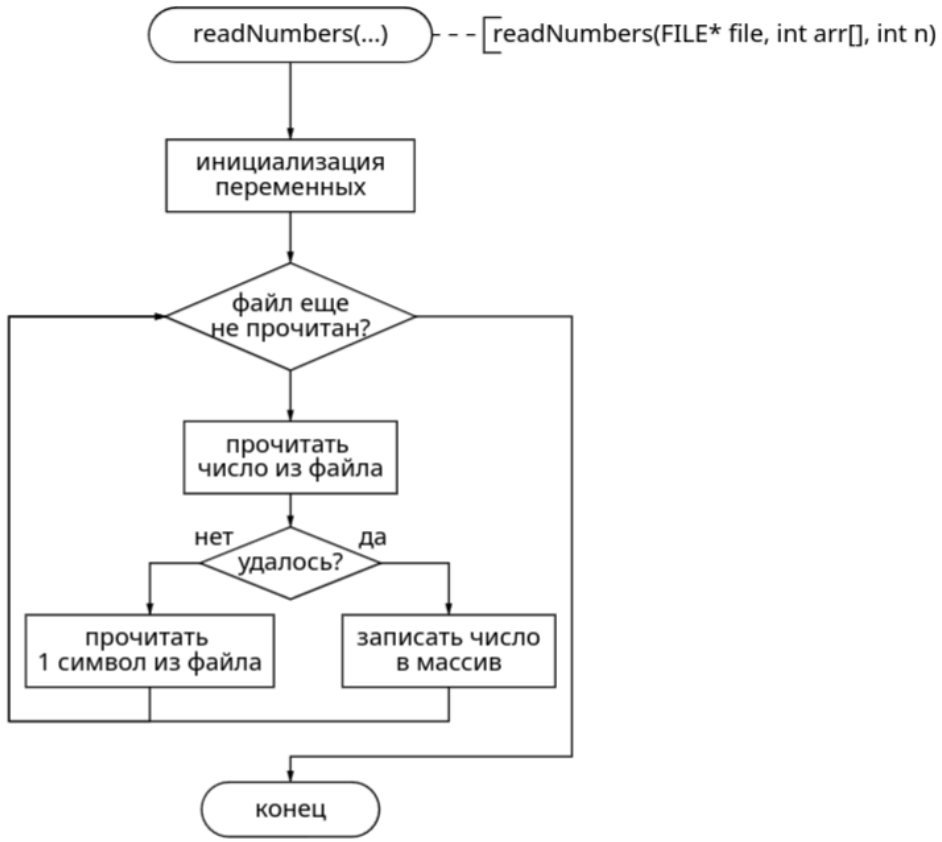
В ходе выполнения лабораторной работы был реализован и исследован алгоритм сортировки бинарными вставками на языке C. Основные выводы:

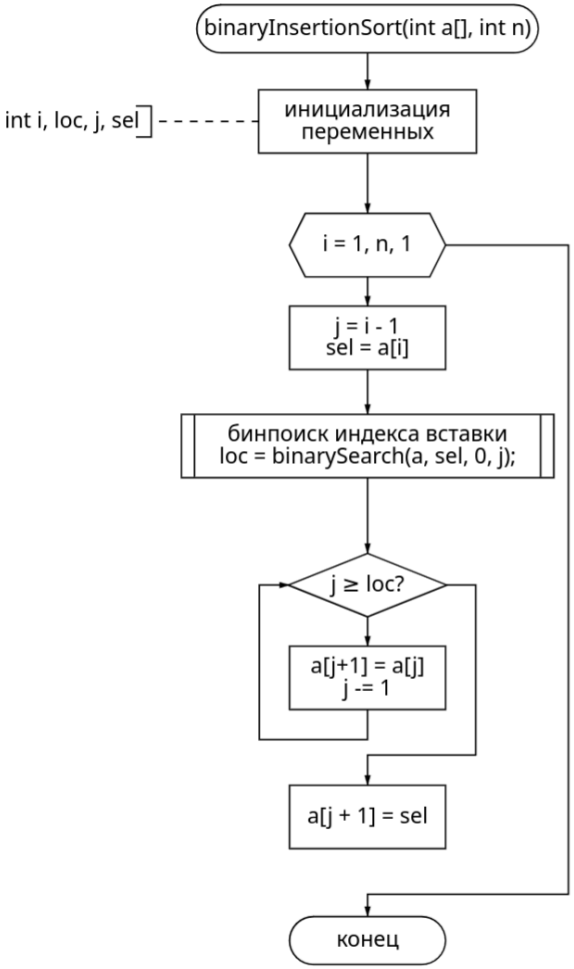
1. Использование бинарного поиска позволяет оптимизировать процесс нахождения позиции элемента до $O(\log n)$, однако общая сложность остается квадратичной $O(n^2)$ из-за необходимости сдвига элементов в памяти.
2. Реализованная программа корректно работает как со статическим, так и с динамическим выделением памяти.
3. Динамическое выделение памяти является более предпочтительным для работы с файлами неизвестного объема, так как позволяет избежать переполнения буфера или нерационального использования ресурсов.

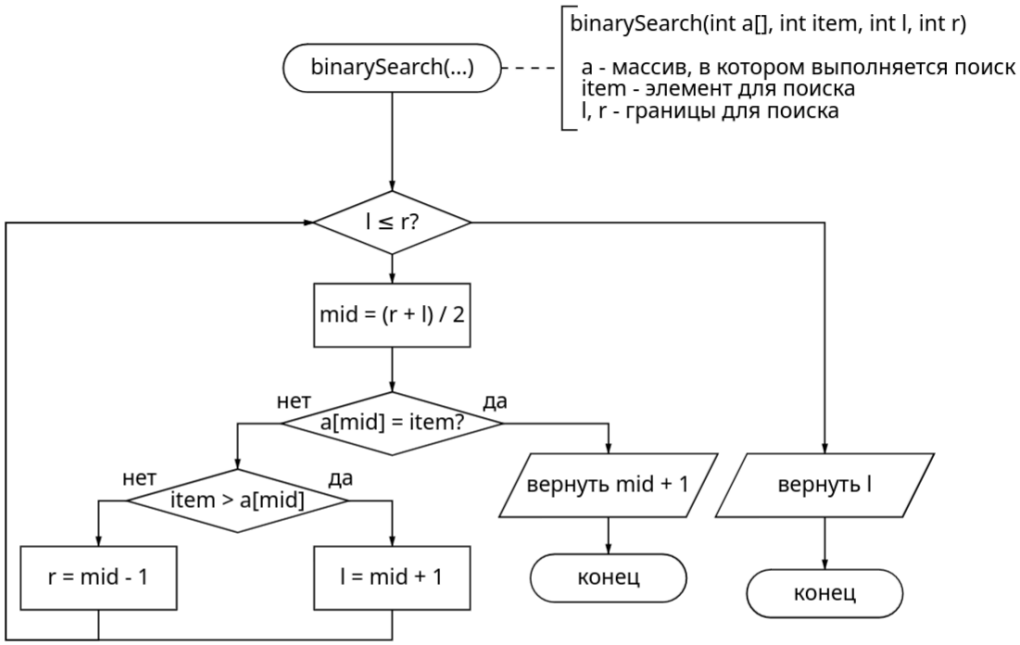
ПРИЛОЖЕНИЕ А




countNumbers(...)
countNumbers(FILE* file)
инициализация переменных
файл еще не прочитан?
прочитать число из файла
нет
да
удалось?
прочитать 1 символ из файла
count += 1
вернуть count
конец


readNumbers(...)
readNumbers(FILE* file, int arr[], int n)
инициализация переменных
файл еще не прочитан?
прочитать число из файла
нет
да
удалось?
прочитать 1 символ из файла
записать число в массив
конец


binaryInsertionSort(int a[], int n)
int i, loc, j, sel
инициализация переменных
i = 1, n, 1
j = i - 1
sel = a[i]
бинпоиск индекса вставки
loc = binarySearch(a, sel, 0, j);
j ≥ loc?
a[j+1] = a[j]
j -= 1
a[j + 1] = sel
конец


binarySearch(int a[], int item, int l, int r)
a - массив, в котором выполняется поиск
item - элемент для поиска
l, r - границы для поиска
binarySearch(...)
l ≤ r?
mid = (r + l) / 2
нет
a[mid] = item?
да
нет
item > a[mid]
да
r = mid - 1
l = mid + 1
вернуть mid + 1
вернуть l
конец
конец